Capítulo 22

SSH

Al terminar este capítulo, entenderás:

- Qué es y qué ofrece SSH.
- Cómo establecer una shell remota.
- Cómo utilizar el fichero `~/.ssh/config`.
- Qué ventajas ofrece la autenticación con clave pública/privada.
- Cómo funciona la autenticación con certificados.
- Cómo copiar archivos de forma segura entre nodos con SCP.

SSH (Secure SHell) es un protocolo de aplicación que permite establecer conexiones con máquinas remotas. Podríamos decir que es una versión mejorada del veterano y muy inseguro `telnet`. Sin embargo, es mucho más que eso: SSH proporciona un canal seguro (cifrado) para ejecutar comandos, transferir archivos[1], redireccionar conexiones, crear túneles que pueden transportar datos de conexiones arbitrarias o simplemente como sistema de autenticación.

22.1. Shell segura

Como ya indica su nombre, el uso más común de la aplicación SSH es el de «shell remota», es decir, un programa que permite abrir una sesión en un computador remoto y ejecutar programas. En este documento vamos a usar la aplicación `OpenSSH`, que en el caso de sistemas GNU/Linux suele estar disponible como dos paquetes: `openssh-server` y `openssh-client`. Por supuesto, existen otras muchas implementaciones para casi cualquier SO.

Se trata, por supuesto, de una aplicación cliente-servidor: El servidor SSH, que debe estar instalado en cada computador que queramos que sea accesible, siempre está a la espera de conexiones, mientras que el cliente SSH es el que inicia la conexión.

[1] con el programa `scp`

Para probar SSH puedes utilizar un contenedor docker cuya configuracióne está en el directorio /ssh-docker. El siguiente comando, que se debe ejecutar dentro de ese directorio, crea la imagen y arranca el contenedor que ejecuta el servidor SSH:

```
alice@maine:~/code/ssh-docker$ docker compose up -d
[...]
```

La imagen contiene un sistema GNU/Linux con un servidor SSH listo para conectar. Abre un terminal en tu sistema y ejecuta el programa ssh, que es el cliente del protocolo.

```
alice@maine:~$ ssh user@172.20.0.2
user@172.20.0.2's password:
user@sidney:~$
```

El argumento del programa incluye el usuario (user) y la dirección IP o el nombre del servidor. Si el puerto es distinto del 22 (valor por defecto) también se puede indicar aquí con otro parámetro.

El programa ssh trata de establecer una conexión segura con el servidor SSH que hay en 172.20.0.2 (el contenedor docker). Este proceso pide la contraseña de sistema asociada a ese usuario, que no se muestra al introducirla. Para este ejemplo la contraseña es **secret**.

22.2. Configuración

Para simplificar el comando de conexión se puede escribir un archivo de configuración llamado ~/.ssh/config en el **computador cliente**. Para el caso anterior, el archivo podría ser algo como:

```
Host sidney
    hostname 172.20.0.2
    User user
```

Es importante señalar que este archivo debe tener permisos de lectura/escritura solo para el usuario (-rw-------) o será ignorado. Una vez tengas esta configuración puedes conectar simplemente con:

```
alice@maine:~$ ssh sidney
user@172.20.0.2's password:
```

Este modo de autenticación con usuario/clave se llama de «clave privada» porque se asume que solamente el usuario conoce dicha clave.

22.3. Acceso con clave pública

Para evitar tener que escribir constantemente la clave cada vez que conectas a una misma máquina, SSH ofrece un sistema de autenticación de «clave pública». Este sistema utiliza un par de claves privada/pública. La clave pública se coloca en el sistema al que quieres acceder y, de hecho, no hay ningún problema en que cualquiera la pueda leer o copiar (por eso es pública). Sin embargo, la clave privada debe quedar custodiada por el usuario y nunca debe ser accesible para nadie más.

Para generar el par de claves ejecuta el siguiente comando aceptando todos los valores por defecto (pulsa `ENTER`):

```
alice@maine:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/alice/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alice/.ssh/id_rsa
Your public key has been saved in /home/alice/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:KXUrUqPxk/EPiXEBfgqK3HlSqvhphSeh6O+qm9tz1DM alice@maine
```

Esto genera dos archivos llamados `id_rsa` (clave privada) y `id_rsa.pub` (clave pública) en el directorio `/home/alice/.ssh/`.

Ahora se debe enviar la clave pública al servidor. Esto se puede hacer fácilmente con:

```
alice@maine:~$ ssh-copy-id sidney
user@172.20.0.2's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'sidney'"
and check to make sure that only the key(s) you wanted were added.
```

Si no se le indica otra cosa, este programa almacena la clave pública del usuario en `~/.ssh/id_rsa.pub`— en el archivo `~/.ssh/authorized_keys` del servidor (`sidney` en este ejemplo).

Y tal como indica, ahora deberíamos poder entrar en la máquina `sidney` simplemente con:

```
alice@maine:~$ ssh sidney
Last login: Mon Mar 14 21:07:15 2022 from 172.20.0.1
user@sidney:~$
```

Esto no solo es interesante porque ahorra al usuario tener que escribir su contraseña constantemente. Además permite que las aplicaciones puedan automatizar tareas sin la intervención de un usuario. Los sistemas de control de versiones (como `git`) o sistemas de Integración o Despliegue Continuo (CI/CD) utilizan habitualmente autenticación SSH con clave pública.

También es posible ejecutar un único comando en el servidor y ver la salida directamente en el computador cliente. Simplemente hay que escribir el comando a continuación:

```
alice@maine:~$ ssh sidney ls /home/
user
```

22.3.1. Agente SSH

Cuando se generan las claves con `ssh-keygen`, existe la opción de proteger la clave privada con una contraseña *(passphrase)*, que en el ejemplo anterior quedó vacía. Esta contraseña evita que otra persona pueda utilizar la clave privada sin permiso, incluso disponiendo del archivo. El inconveniente es que `ssh` la pedirá en cada conexión.

El programa `ssh-agent` resuelve ese problema. Es un proceso que permanece en ejecución durante la sesión del usuario y que mantiene en memoria las claves privadas ya descifradas. La *passphrase* solo se introduce una vez por sesión. La clave se añade al agente con `ssh-add`:

```
alice@maine:~$ ssh-add
Enter passphrase for /home/alice/.ssh/id_rsa:
Identity added: /home/alice/.ssh/id_rsa (alice@maine)
```

La opción `-l` lista las claves cargadas en el agente:

```
alice@maine:~$ ssh-add -l
3072 SHA256:KXUrUqPxk/EPiXEBfgqK3HlSqvhphSeh6O+qm9tz1DM alice@maine (RSA)
```

Los entornos de escritorio habituales arrancan el agente automáticamente al iniciar la sesión. Además, el agente puede «reenviarse» al nodo remoto (opción `ForwardAgent`), lo que permite saltar desde ella a otros servidores utilizando las claves custodiadas en el nodo origen, pero sin copiarlas fuera de él.

22.4. Autenticación con certificado

Otra forma de autenticar un usuario es mediante certificados, una opción algo más avanzada que la autenticación con clave pública de la sección anterior. Estos certificados son emitidos por una autoridad de certificación

o CA (Certificate Authority), que firma (con su clave privada) el par de claves (privada/pública) de los usuarios. El resultado son certificados que permiten autenticar usuarios en hosts y viceversa.

La ventaja de este sistema es que el administrador de un servidor puede expedir certificados para un número arbitrario de usuarios. Un mismo par de claves firmado por un administrador puede otorgar derecho de acceso a múltiples servidores sin tener que modificar su configuración.

Las claves de la CA se crean también con `ssh-keygen`:

```
$ ssh-keygen -t rsa -f ca_key -P "" -C ca_key
Generating public/private rsa key pair.
Your identification has been saved in ca_key
Your public key has been saved in ca_key.pub
The key fingerprint is:
SHA256:nbN+pdyB3paUUnoPe60KJbkC3flKiXCVbSIH51wQXbo ca_key
```

Aunque `ssh-keygen` proporciona una gran cantidad de opciones, en este caso solamente utilizamos algunas de ellas:

- **`-t`** permite indicar el algoritmo (en este caso cifrado RSA).
- **`-f`** especifica el nombre del archivo destino.
- **`-P`** define la contraseña (vacía en este caso).
- **`-C`** añade un comentario que aparece al final de la clave pública.

La clave privada (`ca_key`), por supuesto, debe almacenarse de forma segura, ya que es con la que se firma y emite los certificados. La clave pública (`ca_key.pub`) es la que se utiliza el servidor SSH en el proceso de autenticación de los clientes.

22.4.1. Certificado de usuario

Los certificados SSH de usuario hacen posible autenticar usuarios en un servidor determinado. Para ilustrar cómo funcionan vamos a generar un certificado para el usuario `user` de la máquina `sidney` como lo haría su administrador. Primero creamos un par de claves pública/privada para el usuario, con el nombre `user_key`.

```
$ ssh-keygen -f user_key
```

Después, se firma la clave pública del usuario (`user_key.pub`) con la clave privada de la CA (`ca_key`):

```
$ ssh-keygen -s ca_key -I sidney -n user user_key.pub
```

Con este comando se ha generado también el certificado `user_key-cert.pub`.

Las opciones que se han usado en la firma son:

-s para indicar la clave con la que se quiere firmar *(sign)*.

-I para determinar la identidad del certificado, normalmente el nombre del *host* (puede utilizarse en el futuro para revocar un certificado, por ejemplo).

-n que permite definir una lista de usuarios y/o *hosts* donde el certificado es válido (en este caso únicamente el usuario `user`).

En el servidor, es necesario almacenar la clave pública de la CA con los permisos adecuados (644). En la máquina `sidney`:

```
root@sidney:/etc/ssh# chmod 644 ca_key.pub
root@sidney:/etc/ssh# ls -la ca_key.pub
-rw-r--r-- 1 user user    562 Mar  3 09:13 ca_key.pub
```

Para indicar que todas las claves de usuario firmadas por la CA se pueden autenticar en el servidor, se debe añadir lo siguiente en el archivo de configuración `/etc/ssh/sshd_config`:

```
TrustedUserCAKeys /etc/ssh/ca_key.pub
```

Para que los cambios tengan efecto reiniciamos el servidor SSH (programa `sshd`) con:

```
root@sidney:/home/user# systemctl restart sshd
```

Con esto termina la configuración el servidor (`sidney` en este ejemplo). Este proceso lo realizar automáticamente la construcción de la imagen docker y se puede ver en el archivo `Dockerfile`.

Por otro lado, el cliente tiene que copiar su clave privada (`user_key`) y su certificado (`user_key-cert.pub`) en su directorio `~/.ssh/`:

```
alice@maine:~$ ls -l ~/.ssh
-rw------- 1 alice    alice    2590 mar  3 09:35 user_key
-rw-rw-r-- 1 alice    alice    2020 mar  3 09:40 user_key-cert.pub
-rw-rw-r-- 1 alice    alice     137 mar  3 10:04 config
```

Además, habrá de indicar que quiere usar esa clave a la hora de acceder a la máquina `sidney`:

```
alice@maine:~$ ssh -i ~/.ssh/user_key sidney
```

Para mayor comodidad, puede añadirse la clave a la configuración de `sidney` en `~/.ssh/config`:

```
Host sidney
    hostname 172.20.0.2
    User user
    IdentityFile ~/.ssh/user_key
```

Y con esto se podrá acceder simplemente con:

```
alice@maine:~$ ssh sidney
```

La diferencia respecto a lo explicado en la sección 22.3 es que el usuario nunca necesitó una clave textual para acceder al servidor. De hecho, el servidor se puede configurar para que la única forma de autenticación permitida sea mediante clave pública [2], de modo que no hay posibilidad de usar `ssh-copy-id`. Esto se considera más seguro.

22.5. Copia de archivos con SCP

SCP (Secure Copy Protocol) es un protocolo de copia segura de archivos entre computadores cualesquiera conectados a Internet. Resulta especialmente cómodo ya que el propio servidor de `OpenSSH` lo incorpora y está activado por defecto.

Con la configuración previa, es tan sencillo como:

```
$ scp un-\file sidney:
un-\file                                      100%    5    16.6KB/s   00:00
```

También es posible copiar directorios recursivamente con la opción `-r`.

Y ¿qué más?

Este capítulo ha mostrado SSH como protocolo de shell remota, así como sus esquemas de autenticación —usuario/contraseña, clave pública y certificados firmados por una CA— y también como mecanismo de transferencia segura de archivos.

Sin embargo, el canal cifrado y autenticado que ofrece SSH tiene muchas más aplicaciones: redirección remota de puertos, túneles cifrados o su uso como proxy SOCKS para encaminar tráfico arbitrario a través de una única conexión. Estas capacidades resultan especialmente útiles para acceder a redes privadas o atravesar cortafuegos.

[2] `PasswordAuthentication no`